# 뜨개더

QA PORTFOLIO

## 01 작업 기록을 기준으로 검증한 iOS 앱

유하은 | Tibero DB QA 5년차 | 개인 프로젝트

뜨개더는 도안(뜨개 작업 설명서)을 보며 단수(완료한 줄 수)를 기록하고, 작업 메모와 재료를 관리하는 iOS 앱입니다. 뜨개질을 하며 도안과 용어 검색, 단수 기록을 따로 관리해야 했던 불편함에서 시작했습니다. 도안을 보다가 모르는 기술을 찾아보고, 돌아와 작업을 이어갈 수 있도록 기능을 연결했습니다.

DB QA에서 다루던 기능과 버그 수정 검증을 넘어, 서비스의 기대 동작을 정하고 사용자 흐름에서 문제를 찾아 개선하는 과정을 경험하고자 진행했습니다. 특히 사용자가 남긴 기록을 잃거나, 잘못된 상태를 믿고 작업하는 상황을 우선했습니다.

### 이 프로젝트에서 보여주고 싶은 것

| 대표 사례 | QA의 판단 | 확인 근거 |
| --- | --- | --- |
| 도안 교체 시 드로잉 유실 | 교체 성공과 기록 삭제의 동의를 별도로 판단 | 취소·승인 화면과 자동화 |
| 한 항목의 오류로 목록 전체 실패 | 화면의 이전 값과 실제 조회 실패를 구분 | HTTP 응답과 SQL 저장값 |
| 두 기기 동시 수정 | 서버 선행값과 미저장 입력을 함께 보호 | 충돌 화면, HTTP 409, 최종 DB |

### 프로젝트 구성

SwiftUI 앱은 로컬 저장을 지원하며, 서버 모드에서는 NestJS API와 Prisma·PostgreSQL을 사용합니다. 수동 검증과 Postman·Charles·SQL로 현상을 확인하고, Appium·pytest·Swift 테스트와 GitHub Actions로 반복 확인 범위를 넓혔습니다.

이 문서는 기획과 QA 판단을 중심으로 대표 사례를 설명합니다. 각 사례의 링크에서 당시 조건과 원본 결과를 확인할 수 있습니다.

소스와 실행 방법 / 기능 기준 / 테스트·결함 추적표

# 02 무엇을 먼저, 어떻게 검증했는가

## 작업이 끊기는 지점에서 우선순위를 정했습니다

| 우선 검증 영역 | 위험 | 검증 기준 |
| --- | --- | --- |
| 저장·동기화·계정 전환 | 유일한 기록 유실, 이전 계정 데이터 노출 | 서버·기기 상태 대조, 소유 범위 확인 |
| 도안 교체·앱 종료 | 사용자가 예상하지 못한 기록 삭제 | 취소·복구 경로와 삭제 안내 확인 |
| 화면 이동·목록 갱신 | 작업 위치 초기화, 이전 값을 최신으로 오인 | 이동 전후 맥락과 실제 조회 결과 대조 |

기능별 기대 동작을 정리한 뒤 명세와 코드를 대조했습니다. 앱에서 문제를 재현하면 API 응답, DB 저장값, 기기 저장 상태를 함께 확인해 발생 지점을 좁혔습니다. 결함에는 재현 조건과 기대·실제 결과를 남기고, 수정 확인과 주변 기능의 회귀 검증을 연결했습니다.

## 한 도구로 판단하기 어려운 부분은 다른 관측으로 보완했습니다

| 방법 | 확인한 내용 | 범위 |
| --- | --- | --- |
| 앱 조작·Appium | 교체 취소, 화면 복귀, 충돌 안내와 입력 보존 | 사용 흐름과 화면 상태 |
| Postman·Charles·pytest | 요청·응답, 실패·지연·재시도 조건 | API 계약과 네트워크 조건 |
| SQL·실제 PostgreSQL 시험 | 저장값, 자식 기록, 트랜잭션과 경합 | 화면과 독립된 데이터 확인 |
| Swift 테스트 | 초안 복구, 계정·프로젝트별 저장 격리 | 앱 내부 동작 |

초기 수동 검증은 22건을 설계해 21건을 실행했고, 당시 결과는 11 PASS·10 FAIL이었습니다. 이후 수정 결과는 최초 판정과 구분해 남겼습니다.

## 본인의 역할과 AI 활용

앱 기획, 테스트 우선순위와 기대 결과, 충돌 시 입력 보존 및 직접 재저장 정책을 정했습니다. 기존 수동·Postman·Charles·SQL 수행 이력을 바탕으로 사례를 구성했습니다. AI는 앱·서버 구현, 코드 분석과 후속 수정, 자동화 구현·실행·증거 수집에 활용했습니다. AI가 수행한 후속 실행을 직접 수동 수행한 것으로 기재하지 않았습니다.

수동 실행 원본 / API·네트워크 검증 / 역할과 회차별 조건

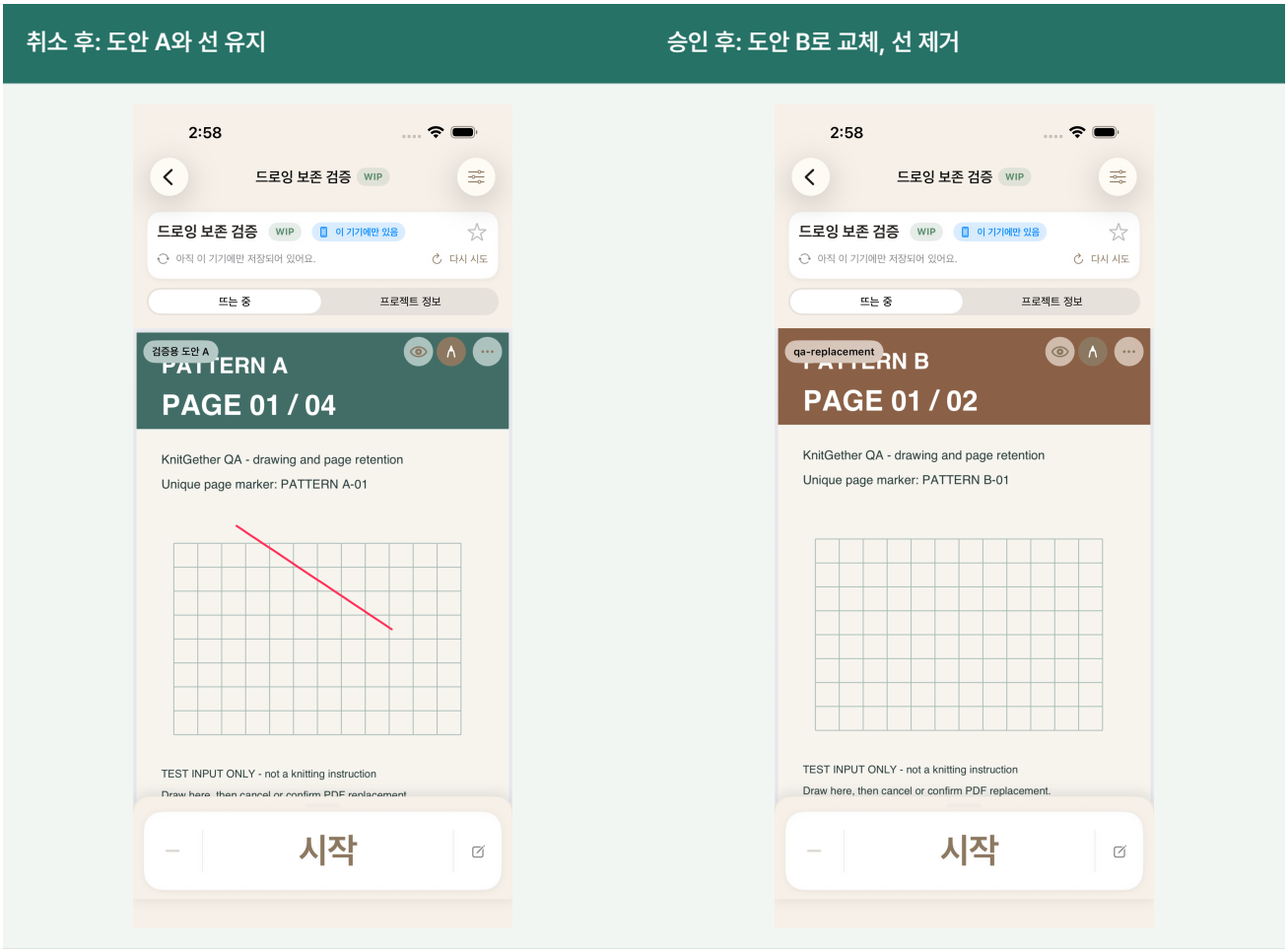
# 03 도안 교체가 성공해도 정상은 아니었습니다

### DEF-10 | 삭제 안내와 취소 시 기록 보존

기존 도안에 드로잉(작업 중 남긴 선 표시)을 남긴 뒤 다른 도안으로 교체하자, 안내나 확인창 없이 선이 사라졌습니다. 교체 자체는 성공했지만 사용자가 남긴 기록이 사전 안내 없이 삭제됐습니다.

교체 전에 드로잉이 삭제된다는 사실을 알리고, 취소하면 기존 도안과 선을 유지하며, 승인했을 때만 교체하도록 검증 기준을 정했습니다. 수정 후에는 다음 두 경로를 나눠 확인했습니다.

| 취소 후: 도안 A와 선 유지 | 승인 후: 도안 B로 교체, 선 제거 |
|---|---|




촬영: 2026-10-04 후속 자동화 실행. 로컬 모드의 PDF 직접 교체 경로입니다. 설치 앱의 소스 커밋은 확정되지 않았으며 서버 동기화의 통과 근거로 사용하지 않습니다.

Appium은 '그리기 삭제' 메뉴의 표시로 드로잉 상태를 간접 검사했고, 실제 선의 유지와 제거는 화면으로 보완했습니다. 자동화의 관측값이 무엇을 증명하는지 확인한 사례입니다.

최초 재현과 수정 기록 / 화면·로그·실행 조건

# 04 목록이 보여도 최신 조회는 실패할 수 있습니다

DEF-08 | 화면·API·DB를 연결한 원인 분석

프로젝트 하나의 단수 정보만 SQL로 제거하자 목록 API 전체가 HTTP 500으로 실패했습니다. 앱에는 이전 목록이 남아 있어 화면만 보면 정상처럼 보였습니다.

정상 프로젝트의 서버 단수를 22에서 99로 바꾼 뒤 새로고침했지만 화면은 22였습니다. DB에는 99가 저장돼 있었고 API는 실패했습니다. 정적 분석 때 예상한 클라이언트 해석 오류보다 앞선, 서버 응답 생성 단계의 문제였습니다.

## 한 항목의 오류가 정상 항목의 조회를 막지 않도록 했습니다

| 확인 지점 | 수정 전 | 수정 후 당시 기록 |
| --- | --- | --- |
| 목록 API | HTTP 500 | HTTP 200, 오류 항목 제외 |
| 정상 프로젝트 화면 | 이전 단수 22 | 변경된 단수 99 |
| 오류 항목 개별 조회 | HTTP 500 | 500 잔존, 후속 과제 |

프로젝트별 응답 변환에서 오류를 분리하도록 수정됐습니다. 목록이 표시되는지만 보지 않고 정상 데이터가 최신 값으로 돌아오는지 확인했습니다.

## 제출 준비 중 회귀 조건도 보완했습니다

기존 자동화는 단수 행을 삭제 상태로만 바꿨습니다. 2026-10-29에는 최초 재현과 같은 **행 삭제**와 **삭제 상태 변경**을 각각 시험했습니다. 두 조건 모두 정상 항목 포함, 오류 항목 제외, SQL로 변경한 단수 99의 반환을 확인했습니다. 이 회차에는 앱 화면을 새로 시험하지 않았습니다.

개별 조회의 500은 현재도 재현됐습니다. 이를 정상 결과로 고정하던 단언을 분리하고, 알려진 실패 1개로 표시했습니다. 복구 응답과 사용자 안내는 남은 개선 범위입니다.

최초 실행 / 보완한 API·SQL 테스트 / 이번 실행 결과

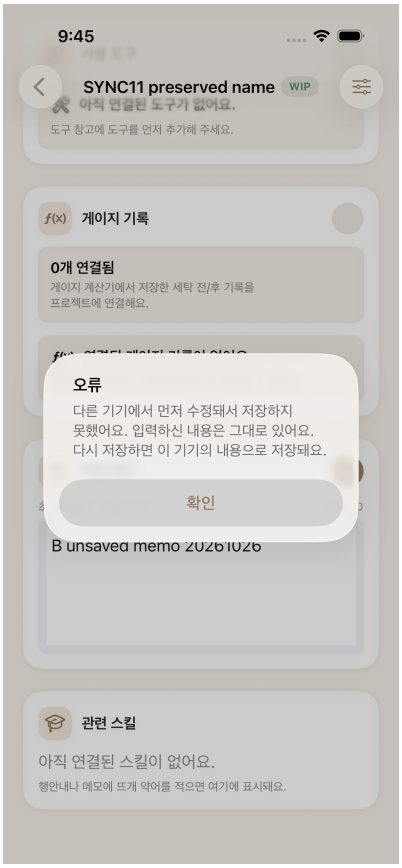
# 05 충돌을 막은 뒤에도 다시 저장할 수 있어야 합니다

### SYNC-11 | 선행 저장과 미저장 입력의 동시 보호

같은 계정의 두 기기가 하나의 프로젝트를 수정하면 오래된 내용이 먼저 저장한 값을 덮을 수 있습니다. 저장을 거절하더라도 작성 중인 메모가 사라진다면 기록을 보호했다고 볼 수 없었습니다.

서버에는 먼저 저장한 값을 남기고, 나중 기기에는 충돌 안내와 미저장 입력을 유지하도록 정했습니다. 재조회 후 사용자가 직접 저장하며, 자동 재저장은 하지 않는 기준입니다. 초안은 계정·프로젝트별 기기에 보관하고 저장 성공 또는 명시적 삭제 시 정리합니다.

| 확인 순서 | 판정 기준과 당시 결과 |
|---|---|
| B 입력 → A 저장 → B 저장 | HTTP 409, DB의 A 메모 유지 |
| B의 충돌 화면 | 안내와 B 입력 유지 |
| B에서 직접 재저장 | DB에 B 메모 저장, 시험한 다른 필드 유지 |
| 앱 종료·재실행 | 초안 복구, 버리기 후 미복구 |




촬영: 2026-10-26 두 시뮬레이터 후속 자동화. 기본 12개는 UI/API 혼합, 보완 7개는 양쪽 변경을 화면에서 수행했습니다. 자동 재저장 없음은 300초 관찰이며, 메모·단수 외 모든 입력의 보존을 보장하지 않습니다.

순서별 화면·HTTP·SQL / 충돌 후 DB / 재저장 후 DB

# 06 메모 충돌에서 연관 데이터까지 확인 범위를 넓혔습니다

## 부분 변경 후 오래된 전체 저장이 들어오는 경우

메모 간 충돌을 막아도 단수·작업 세션·행 안내·도안 전용 API의 변경이 프로젝트 버전에 반영되지 않으면, 오래된 전체 저장이 성공할 수 있습니다. 프로젝트에 속한 연관 데이터(자식 기록)를 지우거나 삭제한 기록을 되살리는 위험입니다.

후속 검증에서는 행 안내 생성·수정·삭제와 PDF·드로잉 변경 뒤 오래된 전체 저장이 HTTP 200으로 성공하는 6개 실패를 재현했습니다. 다른 프로젝트의 자식 ID 재사용 3개 실패도 확인했습니다. 모두 후속 자동화 실행에서 얻은 결과입니다.

| 보완 범위 | 확인 기준 |
| --- | --- |
| 부모 버전과 자식 변경 | 같은 트랜잭션에서 처리, 오래된 전체 저장은 409 |
| 실제 동시 요청 | 같은 기준 버전에서 한 요청만 성공, 패자 기록 유입 없음 |
| 중간 실패 | DB와 기존 파일 바이트 보존 |
| 자식 ID 재사용 | 다른 계정·프로젝트의 기록 수정 차단 |

2026-10-26 수정 후 실제 HTTP·PostgreSQL 40개가 통과했습니다. 2026-10-29에도 같은 회귀 40개를 새 전용 DB에서 재실행해 모두 통과했습니다. DB 조회·쓰기에는 실제 SQL을 사용하고, 읽기 순서와 실패 시점은 테스트 훅으로 제어합니다. 일반 사용자의 장기 부하 시험과는 구분합니다.

## 화면 흐름에서 확장한 기준: 도안 페이지 유지

사전에서 돌아오면 보던 페이지가 유지됐지만 홈 탭을 거치면 1페이지로 초기화됐습니다. 두 경로 모두 작업을 이어가는 흐름으로 보고, 탭 복귀까지 페이지 유지 범위를 넓혔습니다. 실제 3페이지 이동과 고유 표시를 기준으로 두 경로를 확인했습니다. 확대율·페이지 안의 스크롤 위치·앱 재실행 후 복원은 당시 범위에서 제외했습니다.

동시성 시험 코드 / 수정 전후 결과 / 페이지 복귀 증거

# 07 실행 결과는 환경과 계층을 나눠 읽습니다

## 2026-10-29 서버·DB 검증

| 검증 | 결과 | 의미 |
| --- | --- | --- |
| NestJS 서버 빌드 | 성공 | 서버 소스 빌드 |
| 서버 경로·파일 회귀 | 180 PASS | 기존 147개 + 신규 경로 33개 |
| 실제 HTTP·PostgreSQL 동시성 | 40 PASS, 0 SKIP | 저장 경합, 롤백, 자식·파일 보존 |
| API·네트워크·DB | 76 PASS, 알려진 실패 1개 | 목록 결손 2조건 포함, 개별 조회 500 잔존 |

위 로컬 실행 소스는 60c0a70입니다. 서로 겹치는 검증을 전체 E2E 건수로 합산하지 않았습니다. 10월 28일 Windows에서 발견한 파일 경로 구분자 문제는 수정 후 실제 Windows·Linux CI에서 경로 33개씩 통과했습니다. 33개는 서버 180개에 포함됩니다.

## 2026-10-31 iOS 화면 실패 후 재검증

최초 CI의 화면 테스트는 2 PASS·1 FAIL이었습니다. 온보딩에서 계정 화면으로 넘어가지 않았고, 원래 실패를 확인한 자료는 보존했습니다. 시뮬레이터 초기화 후 부팅 준비가 끝날 때까지 기다리고, 탭 직후 목적 화면 도착을 확인하도록 시험 준비와 판정을 보완했습니다.

보완 소스 72bfa8d에서 로컬 UI smoke 3개와 초기화 후 핵심 흐름 재실행이 통과했습니다. 원격 CI도 Swift 359개와 UI smoke 3개를 통과했습니다. 가입·프로젝트·게이지 저장, 오프라인 생성 후 동기화, 캐시 조회를 확인했습니다. 첫 실패의 내부 원인은 미확정으로 남기고, 시험 조건과 후속 결과를 기록했습니다. 이 결과는 제품 전체의 무결함 보장이 아닙니다.

## 이전 실행과 남은 범위

본문의 드로잉·페이지 화면은 10월 4일, 두 기기 충돌 화면은 10월 26일 실행입니다. 10월 26일 단위 정책 UI 24 PASS도 과거 결과입니다. 실물 iPhone, 전 기종, 장기 부하는 이번 통과 범위에 포함하지 않습니다. 속도 향상률·전체 커버리지·운영 성과는 측정하지 않았습니다.

최신 실패 분석·재검증 / 서버·DB 항목별 결과 / 이전 계층별 결과

# 08 이 경험을 서비스 QA로 어떻게 이어갈 것인가

도안 교체가 성공해도 드로잉이 사라질 수 있었고, 목록이 표시돼도 최신 데이터는 아니었습니다. 기능의 실행 여부에서 더 나아가 사용자가 기록을 믿고 작업을 이어갈 수 있는지를 검증 기준으로 삼았습니다.

이 프로젝트에서 화면의 현상을 API 응답과 저장값으로 좁히고, 기대 동작을 취소·충돌·복구 조건까지 구체화했습니다. 수정 후에는 같은 문제의 재발과 주변 데이터의 영향을 함께 확인했습니다. 새로운 서비스에서도 도메인의 규칙을 먼저 이해하고 검증 가능한 기준과 관측 지점으로 바꾸는 데 이 경험을 활용하겠습니다.

## 다음 검증은 사용자 영향이 큰 순서로 진행합니다

| 우선순위 | 남은 검증·개선 | 완료 판단 기준 |
|---|---|---|
| 1 | 오류 항목 개별 조회와 안내 정책 | 500 처리 개선, 복구 가능 여부와 화면 안내 확인 |
| 2 | 충돌 후 재저장 중 통신 실패·계정 전환 | 초안 보존, 중복 저장 없음, 타 계정 노출 없음 |
| 3 | 실제 iPhone의 종료·복귀와 권한 거부 | OS·기기·통신 조건별 결과와 재현 절차 확보 |
| 4 | 데이터 규모별 목록·도안 성능 | 규모와 응답 시간 기준을 정한 뒤 실측 |

시뮬레이터 중심의 개인 프로젝트이며 운영·TestFlight 배포, 전 기종, 장기·대량 경합은 완료 범위에 포함하지 않습니다. 팀 협업과 실제 운영 서비스 경험을 대체하는 이력으로 제시하지 않습니다.

## 근거를 확인하는 순서

기능 기준 → 테스트·RTM → 결함 대장 → 실행 결과

앱 소스 / 서버 소스 / Appium / pytest / 이번 보완의 코드와 증거